TP4 – Diseñando el backend de Mi Backlog

1. Descripción de la aplicación

Mi Backlog es una biblioteca personal de juegos, series y películas. Permite cargar cada título con su tipo, estado (pendiente, en curso, terminado o abandonado), plataforma, género, puntaje y un comentario. También permite marcar favoritos, buscar, filtrar, ordenar, ver cuántos títulos hay en cada estado y exportar/importar la biblioteca en JSON.

Hasta ahora era **solo frontend** (HTML, CSS y JavaScript puro) y guardaba todo en el `localStorage` del navegador. En este TP se diseña su backend y se implementa la primera funcionalidad: **el alta de títulos**.

---

2. Análisis del frontend actual

Funcionalidades principales

Funcionalidad	Dónde está hoy
Agregar un título	`app.js` → `manejarEnvio`
Editar un título	`app.js` → `empezarEdicion` / `manejarEnvio`
Eliminar un título (con confirmación)	`app.js` → `eliminarTitulo`
Marcar / desmarcar favorito	`app.js` → `alternarFavorito`
Buscar, filtrar y ordenar	`filtros.js` → `filtrarYOrdenar`
Contador de títulos por estado	`ui.js` → `renderEstadisticas`
Exportar / importar JSON	`app.js` → `exportar` / `importar`

Entidades

- **Título**: la única entidad con datos propios. Representa un juego, serie o película.
- **Catálogos de valores**: Tipo (Juego, Serie, Película), Estado (Pendiente, En curso, Terminado, Abandonado) y Género (9 opciones). No son entidades con tabla propia, sino listas fijas de valores permitidos.
- **Estadísticas**: no se guardan; se calculan a partir de los títulos.

Datos de un Título

Campo	Tipo	Obligatorio	Quién lo define
`id`	texto	Sí	Sistema
`titulo`	texto (máx. 100)	Sí	Usuario
`tipo`	Juego / Serie / Película	Sí	Usuario
`estado`	Pendiente / En curso / Terminado / Abandonado	Sí (por defecto Pendiente)	Usuario
`plataforma`	texto	No	Usuario
`genero`	lista de 9 géneros	No	Usuario
`puntaje`	entero 1–10	No	Usuario
`comentario`	texto (máx. 500)	No	Usuario
`favorito`	sí/no	No (por defecto no)	Usuario

Campo	Tipo	Obligatorio	Quién lo define
`fechaCreacion`	fecha	Sí	Sistema
`fechaActualizacion`	fecha	Sí	Sistema

Reglas de negocio

1. El nombre es obligatorio, se guarda sin espacios al principio ni al final y tiene como máximo 100 caracteres.
2. El tipo y el estado tienen que ser valores de su lista. Si no se indica estado, es "Pendiente".
3. El género es opcional, pero si se indica tiene que estar en la lista.
4. El puntaje es opcional; si se indica, es un **entero** del 1 al 10.
5. El comentario tiene como máximo 500 caracteres.
6. Un título nuevo nunca es favorito, y su `id` y sus fechas las pone el sistema.
7. Al editar, `fechaCreacion` no cambia y `fechaActualizacion` sí.
8. Importar reemplaza toda la biblioteca, y solo si **todos** los registros del archivo son válidos.

Forma actual de persistencia

- Un array JSON en `localStorage`, con la clave `mi-backlog:titulos` (archivo `storage.js`).
- **Limitaciones:** los datos quedan en un solo navegador y una sola PC. Si se borran los datos del navegador, se pierde todo. Además, todas las validaciones corren en el cliente, y cualquiera puede saltearlas desde la consola del navegador.

---

3. Responsabilidades: frontend vs. backend

Frontend (navegador)	Backend (API)
Mostrar la lista, el formulario, las tarjetas y las estadísticas	Guardar y leer los títulos (persistencia)
Estados visuales: modo edición, avisos, lista vacía, errores al lado de cada campo	**Validar todas las reglas de negocio** (es la validación que vale)
Validación rápida en el formulario, para que el usuario no espere al servidor	Generar el `id` y las fechas; definir los valores por defecto
Buscar, filtrar y ordenar la lista mientras se escribe (ver punto 6.2)	Validar el archivo completo al importar y reemplazar la biblioteca
Confirmaciones ("¿Eliminar…?") y descarga del archivo al exportar	Calcular las estadísticas por estado
Llamar a la API y mostrar sus errores	Devolver códigos HTTP claros (201, 400, 404, 500) y no exponer detalles internos

Criterio: el frontend se ocupa de **cómo se ve y se usa** la app. El backend se ocupa de **qué datos son válidos y dónde se guardan**. La validación aparece en los dos lados a propósito: en el frontend da una respuesta rápida al usuario y en el backend protege los datos.

---

4. Endpoints propuestos

Base: `http://localhost:5080/api`

Método	Ruta	Para qué	Respuestas	Estado
`GET`	`/titulos`	Listar todos (más recientes primero)	200	✅ Implementado

Método	Ruta	Para qué	Respuestas	Estado
`GET`	`/titulos/{id}`	Ver un título	200 / 404	✅ Implementado
`POST`	`/titulos`	Crear un título	201 + `Location` / 400 con errores por campo	✅ Implementado
`PUT`	`/titulos/{id}`	Editar un título completo	200 / 400 / 404	Pendiente
`PATCH`	`/titulos/{id}/favorito`	Marcar o desmarcar favorito. Body: `{ "favorito": true }`	200 / 404	Pendiente
`DELETE`	`/titulos/{id}`	Eliminar un título	204 / 404	Pendiente
`GET`	`/estadisticas`	Cantidad de títulos por estado	200	Pendiente
`PUT`	`/titulos`	Importar: reemplaza la biblioteca completa (todo o nada)	200 / 400	Pendiente
`GET`	`/opciones`	Listas de tipos, estados y géneros, para que el frontend no las tenga repetidas	200	Pendiente

Exportar no necesita un endpoint propio: es `GET /titulos`, y el frontend guarda la respuesta como archivo.

Formato de error de validación (400):

```
{ "errores": { "titulo": "El nombre es obligatorio.", "puntaje": "El puntaje tiene que ser un número entero del 1 al 10." } }
```

Usa los mismos nombres de campo que el formulario, así el frontend puede mostrar cada error al lado de su campo, como hace hoy.

---

5. Arquitectura elegida

Tecnología: C# con ASP.NET Core (.NET 10), API REST que recibe y devuelve JSON. Los datos se guardan en un archivo JSON.

Capas: Controller → Service → Repository.

```
Petición HTTP
    │
    ▼
TitulosController   Recibe la petición y responde con el código HTTP correcto.
    │               No tiene reglas de negocio ni sabe cómo se guardan los datos.
    ▼
TitulosService      Reglas de negocio: valida (ReglasTitulo), genera id y fechas,
    │               aplica valores por defecto. No sabe nada de HTTP ni de archivos.
    ▼
ITitulosRepository  Interfaz de acceso a datos.
    │
    ▼
JsonTitulosRepository   Implementación actual: lee y escribe Data/titulos.json.
```

Por qué la elegí: cada capa tiene una sola responsabilidad. El Controller atiende HTTP, el Service maneja las reglas y el Repository accede a los datos. Así:

- Las reglas de negocio quedan en un solo lugar (`ReglasTitulo` + `TitulosService`), y se pueden probar sin levantar el servidor.
- El Service depende de la **interfaz** `ITitulosRepository`, no del archivo JSON. Para pasar a una base de datos alcanza con escribir otra clase que implemente la interfaz y cambiar una línea en `Program.cs`, sin tocar el Controller ni el Service.
- Los datos que entran (`TituloRequest`) están separados de la entidad guardada (`Titulo`). Por eso el cliente **no puede** mandar `id`, `favorito` ni fechas: esos campos no existen en el DTO de entrada.

Por qué un archivo JSON y no una base de datos (por ahora): es una app de un solo usuario con unos cientos de registros. Un archivo JSON alcanza, no hay que instalar nada y tiene el mismo formato que el export del frontend. Gracias al Repository, cambiarlo después es barato.

Estructura de carpetas:

```
mi-backlog-api/
├── Program.cs                        configuración, inyección de dependencias, CORS, manejo de errores
├── appsettings.json                  puerto y ruta del archivo de datos
├── Controllers/TitulosController.cs
├── Services/TitulosService.cs
├── Services/ReglasTitulo.cs          reglas de validación y listas permitidas
├── Repositories/ITitulosRepository.cs
├── Repositories/JsonTitulosRepository.cs
├── Models/Titulo.cs                  entidad
├── Dtos/TituloRequest.cs             lo que puede mandar el cliente
├── Data/titulos.json                 datos (se crea solo)
└── probar-api.ps1                    pruebas de la API
```

---

6. Decisiones de la IA que cuestioné

6.1 Qué hacer si el archivo de datos está corrupto (la principal)

Propuesta de la IA: que el Repository haga lo mismo que el `storage.js` del frontend: si el archivo no se puede leer, devolver una lista vacía y seguir funcionando.

Por qué la cuestioné: en el frontend esa decisión tenía sentido, porque la app no se rompía. En el backend es peligrosa. Si el archivo se corrompe y el Repository devuelve una lista vacía, el **siguiente alta** escribe un archivo nuevo con un solo título y **borra todos los datos anteriores**, sin ningún aviso. Para el usuario, la biblioteca "desaparece".

Decisión final: si el JSON no es válido, el Repository lanza `DatosCorruptosException`. La API responde `500` con el mensaje "No se pudieron leer los datos guardados. No se modificó nada." y **no escribe nada**, así el archivo se puede revisar y recuperar. También se pidió que la escritura sea segura: primero se escribe un archivo temporal y después se reemplaza el original, para que un corte a mitad de camino no deje el archivo a medias. Está probado en la sección 8.

6.2 Mover la búsqueda y los filtros al backend

Propuesta de la IA: que `GET /titulos` reciba los filtros como parámetros (`?texto=&tipo=&estado=&orden=`) y que el backend filtre y ordene.

Por qué la cuestioné: la búsqueda filtra **mientras el usuario escribe**. Con esa propuesta, cada tecla sería una petición al servidor, con demora y más complejidad. Para unos cientos de títulos, filtrar en el navegador es instantáneo. Además, la lógica ya está en `filtros.js`, separada y con 18 tests (Clase 10).

Decisión final: el backend devuelve la lista completa y el frontend sigue filtrando y ordenando. Filtrar es una cuestión de **presentación**, no una regla de negocio. Si la biblioteca creciera a miles de títulos, se reconsideraría

(paginación y filtros en el servidor).

---

7. Funcionalidad implementada: alta de títulos

Endpoints: `POST /api/titulos`, más `GET /api/titulos` y `GET /api/titulos/{id}` para poder comprobar el resultado.

Controller: solo traduce el resultado a HTTP:

```
[HttpPost]
public async Task<ActionResult<Titulo>> Crear(TituloRequest datos)
{
    var resultado = await servicio.CrearAsync(datos);
    if (!resultado.Exito) return BadRequest(new { errores = resultado.Errores });
    return CreatedAtAction(nameof(Obtener), new { id = resultado.Titulo!.Id }, resultado.Titulo);
}
```

Service: valida, aplica valores por defecto y genera los datos del sistema:

```
public async Task<ResultadoCreacion> CrearAsync(TituloRequest datos)
{
    var errores = ReglasTitulo.Validar(datos);
    if (errores.Count > 0) return new ResultadoCreacion(null, errores);

    var ahora = reloj.GetUtcNow();
    var titulo = new Titulo
    {
        Id = Guid.NewGuid().ToString(),
        Nombre = ReglasTitulo.Limpiar(datos.Titulo),
        Tipo = datos.Tipo!,
        Estado = datos.Estado ?? "Pendiente",
        Plataforma = ReglasTitulo.Limpiar(datos.Plataforma),
        Genero = ReglasTitulo.Limpiar(datos.Genero),
        Puntaje = datos.Puntaje is decimal p ? (int)p : null,
        Comentario = ReglasTitulo.Limpiar(datos.Comentario),
        Favorito = false,
        FechaCreacion = ahora,
        FechaActualizacion = ahora
    };

    await repositorio.AgregarAsync(titulo);
    return new ResultadoCreacion(titulo, errores);
}
```

Repository: guarda en el archivo con un candado, para que dos altas simultáneas no se pisen:

```
public async Task AgregarAsync(Titulo titulo)
{
    await _candado.WaitAsync();
    try
    {
        var titulos = await LeerAsync();
        titulos.Add(titulo);
        await EscribirAsync(titulos);
```

```
    }
    finally { _candado.Release(); }
}
```

El código completo está en la carpeta `mi-backlog-api`.

---

8. Evidencia de que funciona

Pruebas de la API (`probar-api.ps1`)

El script hace 15 peticiones reales a la API corriendo y compara el código HTTP y el contenido de cada respuesta con lo esperado.

```
=== Alta de títulos: POST http://localhost:5080/api/titulos ===

[OK  ] Crear un título válido -> HTTP 201 (esperado 201)
        {"id":"ae7983e1-...","titulo":"Hollow
Knight","tipo":"Juego","estado":"Terminado","plataforma":"Steam",
         "genero":"Aventura","puntaje":9,"comentario":"Excelente","favorito":false,"fechaCreacion":"2026-11-
11T17:19:31...", ...}
[OK  ] Crear con lo mínimo (estado por defecto Pendiente) -> HTTP 201 (esperado 201)
[OK  ] Ignora id, favorito y fecha enviados por el cliente -> HTTP 201 (esperado 201)
[OK  ] Sin nombre -> HTTP 400 (esperado 400)
        {"errores":{"titulo":"El nombre es obligatorio."}}
[OK  ] Nombre con solo espacios -> HTTP 400 (esperado 400)
        {"errores":{"titulo":"El nombre es obligatorio."}}
[OK  ] Tipo y estado que no existen (dos errores juntos) -> HTTP 400 (esperado 400)
        {"errores":{"tipo":"El tipo tiene que ser uno de: Juego, Serie, Película.",
                    "estado":"El estado tiene que ser uno de: Pendiente, En curso, Terminado, Abandonado."}}
[OK  ] Puntaje decimal -> HTTP 400 (esperado 400)
        {"errores":{"puntaje":"El puntaje tiene que ser un número entero del 1 al 10."}}
[OK  ] Puntaje 0 (debajo del mínimo) -> HTTP 400 (esperado 400)
[OK  ] Puntaje 11 (arriba del máximo) -> HTTP 400 (esperado 400)
[OK  ] Puntaje 10 (máximo permitido) -> HTTP 201 (esperado 201)
[OK  ] Nombre de 100 caracteres (justo el máximo) -> HTTP 201 (esperado 201)
[OK  ] Nombre de 101 caracteres -> HTTP 400 (esperado 400)
        {"errores":{"titulo":"El nombre puede tener hasta 100 caracteres."}}

=== Consultas ===

[OK  ] Obtener por id el título creado -> HTTP 200 (esperado 200)
[OK  ] Obtener un id que no existe -> HTTP 404 (esperado 404)
        {"error":"No existe un título con ese id."}
[OK  ] Listar: hay 5 títulos (solo los válidos se guardaron) -> HTTP 200 (esperado 200)

Resultado: 15 pasaron, 0 fallaron
```

En el caso "Ignora id, favorito y fecha" el cliente mandó `id: "mi-id"`, `favorito: true` y una fecha del año 2000. La API generó su propio id, guardó `favorito: false` y puso la fecha actual.

Los datos persisten al reiniciar el servidor

Se detuvo el servidor, se volvió a iniciar y se consultó la lista:

```
GET /api/titulos -> HTTP 200, 5 títulos: xxxxxxxxxxxxxxxxxxx | Puntaje diez | Intento de trampa | Película mínima | Hollow Knight
```

Archivo corrupto: no se pierden datos (decisión 6.1)

Se rompió a propósito el archivo `Data/titulos.json` (JSON cortado a la mitad) y se intentó leer y crear:

```
GET  /api/titulos -> HTTP 500 {"error":"No se pudieron leer los datos guardados. No se modificó nada."}
POST /api/titulos -> HTTP 500 {"error":"No se pudieron leer los datos guardados. No se modificó nada."}
Archivo sin modificar después del POST: True
```

El POST **no sobrescribió** el archivo: el contenido quedó idéntico al anterior (se comparó su hash). Con la propuesta original de la IA, ese POST habría borrado todo.

---

9. Cómo correrlo

Desde la carpeta `mi-backlog-api`:

```
dotnet run
```

La API queda en `http://localhost:5080/api/titulos`. Con la API corriendo, en otra terminal:

```
powershell -ExecutionPolicy Bypass -File probar-api.ps1
```

10. Próximos pasos

1. Implementar el resto de los endpoints (editar, favorito, eliminar, estadísticas, importar, opciones).
2. Conectar el frontend: reemplazar `storage.js` (localStorage) por llamadas `fetch` a la API y mostrar los errores 400 al lado de cada campo.
3. Agregar tests unitarios de `ReglasTitulo` y `TitulosService`, usando un Repository en memoria.